

Aula 36: WebSockets no Backend - Teoria e Prática

Duração: 4 horas (2h teoria + 2h prática)

Parte Teórica (2h) - Fundamentos de WebSockets e Spring WebFlux

1. O que são WebSockets?

- **Comunicação Bidirecional:** Diferente do HTTP (request-response), WebSockets mantêm uma conexão aberta entre cliente e servidor.
- **Tempo Real:** Ideal para chats, notificações, dashboards dinâmicos.
- **Protocolo WS:** `ws://` (não criptografado) ou `wss://` (criptografado).

2. STOMP (Simple Text Oriented Messaging Protocol)

- Subprotocolo sobre WebSockets para troca de mensagens estruturadas.
- **Componentes Principais:**
 - **Broker:** Gerencia tópicos (canais de mensagens).
 - **Clientes:** Envia/recebem mensagens via tópicos.
 - **@MessageMapping:** Define endpoints para mensagens.

3. Spring WebFlux + WebSockets

- **Vantagens:**
 - **Reativo:** Escalável com muitos clientes simultâneos.
 - **Integração Nativa:** Suporte a STOMP e RSocket.

4. Fluxo de uma Aplicação de Chat

1. Cliente se conecta ao endpoint `/ws-chat`.
2. Inscreve-se em um tópico (ex: `/topic/public`).
3. Envia mensagens para `/app/chat.send` (processadas pelo backend).
4. Servidor faz broadcast para todos inscritos no tópico.

Parte Prática (2h) - Implementando o Backend do Chat

1. Configuração do WebSocket (1h)

Arquivo: WebSocketConfig.java

```
java

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws-chat") // Endpoint para conexão
            .setAllowedOrigins("*"); // Permite qualquer origem (em dev)
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic"); // Tópicos para broadcast
        registry.setApplicationDestinationPrefixes("/app"); // Prefixo para métodos @MessageMapping
    }
}
```

2. Controller de Mensagens (1h)

Arquivo: ChatController.java

```
java

import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.messaging.handler.annotation.SendTo;
import org.springframework.stereotype.Controller;
import reactor.core.publisher.Mono;

@Controller
public class ChatController {

    @MessageMapping("/chat.send") // Recebe mensagens enviadas para "/app/chat.send"
    @SendTo("/topic/public") // Retransmite para todos inscritos em "/topic/public"
    public Mono<Message> sendMessage(Message message) {
        return Mono.just(message)
            .doOnNext(msg -> msg.setTimestamp(Instant.now()));
    }
}
```

```
}  
}
```

3. Testando com Postman (Atividade Prática)

1. Configure o Postman para WebSockets:

- o Abra uma nova requisição WebSocket (`ws://localhost:8080/ws-chat`).
- o Envie uma mensagem STOMP:

```
json  
  
{  
  "destination": "/app/chat.send",  
  "content": "Olá WebSockets!",  
  "sender": "Postman"  
}
```

2. Verifique o Broadcast:

- o Inscreva-se no tópico `/topic/public` e veja as respostas.

Exercício Prático para Fixação

Desafio: Modifique o `ChatController` para:

1. Validar mensagens vazias (retornar erro se `content` estiver vazio).
2. Adicionar um tópico privado (`/topic/private.{userId}`) para mensagens diretas.

Solução Sugerida:

```
java  
  
@PostMapping("/chat.private.{userId}")  
@SendTo("/topic/private.{userId}")  
public Mono<Message> sendPrivateMessage(  
    @DestinationVariable String userId,  
    Message message  
) {  
    return Mono.just(message)  
        .filter(msg -> !msg.getContent().isBlank())  
        .switchIfEmpty(Mono.error(new IllegalArgumentException("Mensagem vazia!")));  
}
```

Recursos Adicionais

- Documentação Spring WebSockets: <https://spring.io/guides/gs/messaging-stomp-websocket/>
- STOMP JS (Cliente JavaScript): <https://stomp-js.github.io/>

Próxima Aula: Frontend do Chat com Vue 3 + Quasar! 🚀